Aula 1 – Introdução ao GIT

1. O que é Gerenciamento de Configuração

Gerenciamento de Configuração é o processo de identificar, controlar, rastrear e auditar alterações em artefatos de software ao longo do seu ciclo de vida.

2. Controle de Versões

Controle de versões permite registrar cada alteração realizada no código, identificando:

- *Autor*
- *Data*
- *Alteração realizada*
- *Justificativa*

3. Histórico, Rastreabilidade e Auditoria

Histórico → linha do tempo das alterações

Rastreabilidade → saber quem fez o quê

Auditoria → validar integridade do software

Conecte com mercado:

- Empresas auditadas
- Projetos bancários
- Sistemas hospitalares
- LGPD

4. Ferramentas de Mercado

- Git
- GitHub
- GitLab
- Bitbucket
- Azure DevOps

➔ Sistema de versionamento

➔ Plataforma de hospedagem

5. Problemas Reais por Falta de Versionamento

- Empresa que perdeu código após formatação de máquina
- Desenvolvedor que saiu da empresa levando código
- Deploy com versão errada
- Bug impossível de rastrear

Qual o impacto financeiro disso?

Principais comandos:

🚀 Iniciando e Clonando Repositórios

git init: Inicializa um novo repositório Git local na pasta atual.

git clone <url>: Baixa um projeto existente do GitHub/GitLab para sua máquina.

Principais comandos:

📝 Trabalhando com Alterações

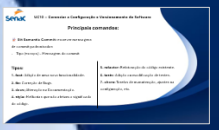
git status: Exibe quais arquivos foram modificados, adicionados ou removidos.

git add <arquivo>: Adiciona um arquivo específico à área de preparo (staging).

git add .: Adiciona todas as alterações do diretório atual à área de preparo.

git commit -m "mensagem": Grava as alterações preparadas (snapshot) no repositório com uma mensagem descritiva.

git diff: Mostra as diferenças exatas entre os arquivos na área de trabalho e o último commit.



Principais comandos:

🌿 **Branches (Ramos)**

- **git branch:** Lista, cria ou exclui branches.
- **git branch <nome>:** Cria uma nova branch.
- **git checkout <nome>:** Alterna para a branch especificada.
- **git checkout -b <nome>:** Cria uma nova branch e muda para ela imediatamente.
- **git merge <branch>:** Mescla as alterações da branch especificada na branch atual.

Principais comandos:

Remoto e Repositórios

- **git push origin <branch>:** Envia os commits locais da branch para o repositório remoto.
- **git pull:** Busca e mescla as alterações do repositório remoto para o local.
- **git fetch: Busca** as alterações remotas, mas não as aplica automaticamente no seu código.

Principais comandos:

🔍 Histórico e Desfazendo Alterações

- **git log:** Mostra o histórico de commits.
- **git log --oneline:** Mostra o histórico de commits de forma compacta.
- **git reset:** Move o HEAD para um estado anterior, desfazendo commits.
- **git revert <commit>:** Cria um novo commit que desfaz as alterações de um commit anterior específico.
- **git restore <arquivo>:** Restaura um arquivo no diretório de trabalho para o estado do último commit.

Principais comandos:

🛠️ Configuração Inicial

- **git config --global user.name "Seu Nome":** Define o nome de usuário.
- **git config --global user.email "seuemail@exemplo.com":** Define o e-mail do usuário.

Obrigado!

Prof. Éder Silva

Senac